

Quicksort algorithm.

Partitioning Around the pivot.

Key idea: partition array around a pivot element.

- pick element of array 3 | 8 | 2 | 5 | 1 | 4 | 7 | 6
pivot.

- rearrange array so that

- left of pivot $\Rightarrow$ less than pivot.

- right of pivot $\Rightarrow$ greater than pivot.

2 | 1 | 3 | 6 | 7 | 4 | 5 | 8
 $\leq$ pivot $\qquad \qquad \qquad >$ pivot.

notes: puts pivot in its "rightful position".

Two cool facts about partition.

1) linear ($O(n)$) time, no extra memory

2) reduces problem size

Quicksort :- High level desc.

Quicksort (array A, length n)

If $n = 1$ return

- $p = \text{choose_pivot}(A, n)$

- partition around P $\boxed{\leq P} \mid P \mid \boxed{\geq P}$
- recursively sort
1st part 2nd part
- recursively sort
1st part 2nd part

Note: using $O(n)$ extra memory, easy to partition around pivot in $O(n)$ time.

$\boxed{3} \mid \boxed{8} \mid \boxed{2} \mid \boxed{5} \mid \boxed{1} \mid \boxed{4} \mid \boxed{7} \mid \boxed{6}$
 pivot $\uparrow \rightarrow$

$\boxed{2} \mid \boxed{1} \mid \boxed{3} \mid \boxed{6} \mid \boxed{7} \mid \boxed{4} \mid \boxed{5} \mid \boxed{8}$

$\boxed{2} \mid \boxed{1} \mid \boxed{3}$

$\boxed{6} \mid \boxed{7} \mid \boxed{4} \mid \boxed{5} \mid \boxed{8}$

Assume: pivot = 1st element of arr.
 [if not, swap pivot $\leftrightarrow$ 1st element as preprocessing step]
 High level idea:-

$\boxed{P} \mid \boxed{\leq P} \mid \boxed{\geq P} \mid \boxed{\text{rest of array}}$

already partitioned | unpartitioned

- single scan through array

- partition around P $\boxed{\leq P} \mid P \mid \boxed{\geq P}$
- recursively sort
1st part 2nd part
- recursively sort
1st part 2nd part

Note: using $O(n)$ extra memory, easy to partition around pivot in $O(n)$ time.

$\boxed{3} \mid \boxed{8} \mid \boxed{2} \mid \boxed{5} \mid \boxed{1} \mid \boxed{4} \mid \boxed{7} \mid \boxed{6}$
 pivot $\uparrow \rightarrow$

$\boxed{2} \mid \boxed{1} \mid \boxed{3} \mid \boxed{6} \mid \boxed{7} \mid \boxed{4} \mid \boxed{5} \mid \boxed{8}$

$\boxed{2} \mid \boxed{1} \mid \boxed{3}$

$\boxed{6} \mid \boxed{7} \mid \boxed{4} \mid \boxed{5} \mid \boxed{8}$

Assume: pivot = 1st element of arr.
 [if not, swap pivot $\leftrightarrow$ 1st element as preprocessing step]
 High level idea:-

$\boxed{P} \mid \boxed{\leq P} \mid \boxed{\geq P} \mid \boxed{\text{rest}}$

already partitioned | unpartitioned

- single scan through array

A Decomposition Principle.

Quicksort theorem: for every input array of length n the average running time the quicksort (with random pivots) is $O(n \log n)$.

Preliminaries:

fix input array A of length n

sample space Ω = all possible outcomes of random choices in quicksort (i.e. pivot sequences)

Key random var: for $\sigma \in \Omega$,

$C(\sigma) = \text{no. of comparisons bet}^n \text{ two input elements made by QS.}$
(given random choices σ)

Goal: $E[C] = O(n \log n)$

Note: can't apply master method

[random, unbalanced subproblems]

[A = fixed input arr]

notation: $Z_i = i^{\text{th}}$ smallest element A

(after sorted) $6 \mid 8 \mid 10 \mid 2$

$Z_2 \quad Z_3 \quad Z_4 \quad Z_1$

for $\sigma \in \Sigma$, indices $i < j$, let
 $X_{ij}(\sigma) = \text{no. of times } z_i, z_j \text{ get}$
 compared in quicksort with pivot-
 sequence σ .

So,

$C(\sigma) = \# \text{ of comparisons bet}^n \text{ input}$

$X_{ij}(\sigma) = 1 \text{ if } z_i \text{ and } z_j \text{ are compared}$

$$\text{Thus, } C(\sigma) = \sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij}(\sigma)$$

By linearity of expectation

$$E[C] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n E[X_{ij}]$$

Since $E[X_{ij}] = 0 \cdot \Pr[X_{ij}=0] + 1 \cdot \Pr[X_{ij}=1]$

$$= \Pr[X_{ij}=1]$$

$$= \Pr[z_i, z_j \text{ get compared}]$$

$$\text{Thus, } E[C] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n \Pr[z_i, z_j \text{ get compared}]$$

General principle

1. Identify random variable that you're really care about.

2. Express it as sum of ind. random var.

3. Apply linearity of expectation.

Analysis: The key insight.

$C(\sigma) = \# \text{ of comparisons QuickSort makes with pivots } \sigma$

$X_{ij}(\sigma) = \# \text{ of times } z_i \text{ \& } z_j \text{ get compared.}$

Recall: $E(C) = \sum_{i=1}^{n-1} \sum_{j=i+1}^n \Pr[X_{ij} = 1] =$

$\Pr[z_i, z_j \text{ get compared}]$

Key claim: $\forall i < j, \Pr[z_i, z_j \text{ get compared}] = \frac{2}{j-i+1}$

Proof: [Key insight]

(Fix z_i, z_j with $i < j$.)

Consider the set $z_i, z_{i+1}, \dots, z_{j-1}, z_j$.

Inductively: as long as none of these

are chosen as a pivot, all are pass-

ed to the same recursive call.

Consider the first among $z_i, z_{i+1},$

$\dots, z_{j-1}, z_j$ that gets chosen as a pivot.

① if z_i or z_j gets chosen first, then z_i and z_j get compared.

② if one of $z_{i+1}, \dots, z_{j-1}$ gets

chosen first, the z_i and z_j are never compared.

① z_i or z_j chosen first $\Rightarrow$ they get compared.

② one of $z_{i+1}, \dots, z_{j-1}$ chosen first $\Rightarrow z_i, z_j$ never compared.

Note: Since pivots always chosen uniformly at random, each of $z_i, z_{i+1}, \dots, z_{j-1}, z_j$ is equally likely to be first.

$$\Rightarrow \Pr[z_i, z_j \text{ get compared}] = \frac{2}{j-i+1}$$

$$\text{So, } E[C] = \sum_{i=1}^n \sum_{j=i+1}^n \frac{2}{j-i+1}$$

For each i , the inner sum is

for each i , the inner sum is

for each i , the inner sum is

for each i , the inner sum is

for each i , the inner sum is

for each i , the inner sum is

for each i , the inner sum is

for each i , the inner sum is

for each i , the inner sum is

completing the proof.

$$E[C] \leq 2n \sum_{k=1}^n \frac{1}{k}$$

claim: $\sum_{k=1}^n \frac{1}{k} \leq \ln n$

Proof :-

so.

$$\sum_{k=1}^n \frac{1}{k} \leq \int_1^n \frac{1}{x} dx$$

$$= \ln n$$

$$= \ln n - \ln 1$$

$$= \ln n \text{ (proved)}$$

